

memor: architecture and product review

Reviewer framing: product architect, LLM memory and context optimisation. **Date:** 2026-08-22. **Method:** five parallel research agents (architecture, memory quality, competitive landscape, product positioning, context-engineering frontier), plus direct measurement against 2,465 local Claude transcripts (82.5M tokens) and the live memor database (33,970 artifacts, 4,133 recalls, 657 adjudicated recall-artifact pairs).

Every claim cites a `path:line`, a command and its output, or a URL with the date read. Where a number is a lower bound or an estimate, it says so. Two claims in the first draft of this review were wrong; both are corrected in place with the error described, because how they failed is diagnostic.

1. The product being reviewed

memor was conceived as **a common long-term memory layer for every LLM tool on a machine** — one store, so context follows the user between tools, and nothing has to be re-established when they switch.

That is not what the README sells. The README leads with compression percentages, and the first draft of this review consequently graded the product as a compression tool. That was a category error on my part, and it produced a roadmap aimed at the wrong target. This version grades memor against its actual thesis.

The verdict changes completely under the correct framing. As a compression product, memor is capped and near-finished. As a shared memory layer, it is built, working, and structurally unable to prove it.

2. Was the shared layer delivered? Yes.

2.1 The store is genuinely tool-agnostic

Scope is by **project**, never by agent (`memor/types.py:34-46`). No agent or source filter exists anywhere in the retrieval path — grepped `memor/retrieve/retriever.py` and `memor/store/sqlite_store.py` ; the only `agent ==` comparison is a display-label default at `sqlite_store.py:320` .

Four tools write, four read:

Writers (artifacts)		Readers (recalls)	
<code>claude_code</code>	24,446	<code>claude</code>	3,635
<code>distill</code>	6,930	<code>cursor</code>	427
<code>goose</code>	1,256	<code>codex</code>	38
<code>jcode</code>	1,249	<code>jcode</code>	31
<code>kimi</code>	95		

2.2 Knowledge actually crosses, in both directions

Joining `recall_outcomes` → `recall_log` → `artifacts` gives the ground truth: which tool *received* an artifact that a *different* tool wrote.

```
claude reads jcode-written artifacts    27
jcode  reads claude_code-written        10
jcode  reads distill-written             7
```

Cross-tool share of served artifact-recalls

Across all projects	5.6% (37 of 657)
---------------------	------------------

In projects where more than one tool is active	28.7% (37 of 129)
--	-------------------

The 5.6% is not a design failure, it is under-exercise. Only 3 of the top 14 projects have two tools writing 20+ chunks; 89% of all writes are `claude_code`. Where two tools genuinely share a project, better than one in four served memories came from the other tool.

The mechanism works. The thesis is delivered.

2.3 What that is worth, and the ledger nobody drew

The product's own numbers, on one line:

```
compression saved : 2,453,627 tokens
recall injected   : 1,146,846 tokens (cost)
-----
net               : 1,306,781 tokens
```

Recall spends 47% of what compression saves. That is a coherent architecture — compression funds the memory layer — and it is a far better description of the product than "11.7% saved on tool output". But it only nets out if a recalled memory is worth more than the ~469 tokens it costs to inject.

The mechanism is plausible on arithmetic: mean recall injection is 469 tokens, while a single `Read` of a 500-line file is roughly 6,000. A recall that averts one file re-read is ~13× cheaper than the derivation it replaces.

Whether recalls actually replace derivations is unmeasured, and that is the central open question of the product, not a detail.

3. Why it looks like a compression product

Because **compression is the only half that can report**.

3.1 The quality loop is hardcoded to one tool

```
memor/daemon.py:367-372 :
```

```
# Feedback + turn metrics: Claude transcripts only
for unit in pending:
    if unit.agent != "claude" or unit.path is None:
        continue
```

`analyze_session_feedback` is the only writer of `use_count`, and it never runs for jcode, cursor, goose or kimi sessions. The consequence is exact and visible in the data:

	used	rejected	unused	pending
same-tool recalls	170	0	354	96
cross-tool recalls	0	0	0	37

Every one of the 37 cross-tool recalls — the product's headline capability — sits `pending` forever. Same-tool recalls are adjudicated and score **100% of judged as used**. Cross-tool recalls are served, then never scored.

The one feature memor exists for is the one feature it cannot measure. That is why a rigorous, measurement-driven project drifted into describing itself by its compression percentage: that number is the one the instrumentation produces.

This is the same failure class the project has corrected seven times in compression — a claim that cannot be falsified — except here it is load-bearing for the entire product thesis.

3.2 The relevance gate is calibrated in the wrong metric

This is the clearest code defect found. It is a units error rather than a mis-tuning, and its practical cost turned out to be smaller than the conversion table alone suggests — the measurement that qualifies it is below, and it reversed my initial reading.

`vec_artifacts` is declared with no distance metric (`memor/store/sqlite_store.py:135-136`), so `sqlite-vec` uses its default, **L2**. Verified from first principles: two orthogonal unit vectors (cosine 0.0) return distance $1.4142 = \sqrt{2}$. The store then computes `sim = 1.0 - float(r["distance"])` (`sqlite_store.py:507`).

1 - L2 is not a cosine. On unit vectors $L2 = \sqrt{(2-2\cos)}$:

true cosine	stored sim	vs the -0.05 gate
+0.90	+0.553	pass
+0.70	+0.225	pass
+0.50	+0.000	pass
+0.40	-0.095	blocked
+0.30	-0.183	blocked
+0.00	-0.414	blocked

The gate at `min_similarity = -0.05` (`memor/recall.py:116`, applied `retriever.py:263`) carries a comment stating the intent: *"Static embeddings put relevant content at >0 and noise at <0, so the default floor is 0.0."* Correct **for cosine**. Applied to `1 - L2`, the effective cut is at **true cosine $\approx +0.449$** — so everything between 0.0 and 0.45 is discarded before fusion, before the memory lane, and before any blended score exists.

That band is exactly where cross-tool matches live. A memory written by jcode about the same subsystem, in different words, is related-but-not-duplicate.

But the gate is better calibrated than the unit error suggests, and this qualifies the finding substantially. Converting the store's `sim` to true cosine and probing the live store with five on-topic queries (150 candidates) against four nonsense queries (112 candidates):

cosine cut	on-topic admitted	nonsense admitted	precision
0.50	11%	0	100%
0.449 (current gate)	~41%	2	~97%
0.40	67%	18	85%
0.35	81%	22	85%
0.30	91%	28	83%
0.25	100%	53	74%

So `-0.05` in L2 units lands at cosine 0.449, which is close to the precision-optimal cut for this embedder. **Whoever tuned it tuned it on behaviour, and arrived somewhere defensible by feel.** The bug is real — the constant means something different from what its comment claims, and cannot be reasoned about or ported to another embedder — but it is not costing the recall disaster the mapping table alone implies.

What it *is* costing is the 0.40–0.449 slice: roughly **26 percentage points of recall for 16 additional false admits**. On this sample the blocked-but-on-topic items are visibly relevant:

```
cos +0.444  BLOCKED  'if the competitors are claiming 20% savings...'
cos +0.429  BLOCKED  'Why are we not able to improve our compression...'
cos +0.421  BLOCKED  'wait so you did identify the reason compression...'
```

The apparent signal-to-noise separation on raw `sim` is only ~ 0.16 , which looks like a weak embedder. It is not: under true cosine, `potion-base-8M` separates matched from nonsense pairs by **0.573**. The model is fine; the unit conversion compresses the usable range into a band too narrow to reason about.

Correction to this review's first draft. I initially reported that the 0.3 blended threshold admits irrelevant-but-fresh memories, since a zero-relevance item scores $0.25 \times 1.0 + 0.10 \times 0.5 = 0.300$. Arithmetically true, practically irrelevant: `norm_rel` is **min-max normalised over the candidate set** (`retriever.py:330`), so the top candidate always scores 1.0. The code says so in a comment I should have read first (`retriever.py:255–260`): *"Min-max normalization forces the top hit to a normalized score of 1.0 ... Gating on raw cosine here is the real relevance filter."* The design anticipated the objection and answered it. The remaining defect is that the filter it points to is in the wrong units.

Predicted and confirmed. If the raw gate is what rejects, then `no_hits` rows should show an empty candidate set rather than a filtered one. All 1,543 `no_hits` rows have `top_score = 0.000` exactly, with no intermediate values.

What this does not explain. Since the gate sits near the precision-optimal cut, it cannot be the whole cause of 40.9% zero-hit. The residual is most likely §3.3: 88% of artifacts have never been recalled, and a store full of scaffolding has little to return however the gate is set.

3.3 What is stored is mostly not knowledge

Of 6,948 memory artifacts (5.67M tokens), **46.3% of tokens are provably ephemeral scaffolding**: 2,316 subagent task prompts ("You are implementing Task 11..."), 293 status reports, plus caveat banners. Sampling the remainder shows more work-log.

The signal that matters: **52.4% of memories ever recalled carry rationale language** ("because", "instead of", "turned out", "root cause") against 44.8% of the corpus. Retrieval is already selecting for rationale — the class that cannot be recovered by reading the repo, and the only class worth carrying between tools.

Anthropic reached the same conclusion independently. Claude Code's auto-memory "**skips anything it can derive from the codebase, such as architecture, file paths, or debugging fixes**", storing `project` memories only for decisions "*that Claude can't derive from the code or git history*". They encoded the thesis as a write-filter. `memor` has no such filter, and **88.0% of its artifacts (29,909) have never been recalled once**.

4. What is genuinely excellent

4.1 The measurement discipline

Seven estimates collapsed under measurement in a single session: diff compressibility 39%→0.1% realised, session-start 30%→4.5%, fetched pages 40%→0.10%, classifier release 5.39%→0.10%, duplicate elision 1.78%→0.53%, superseded reads 15.77%→0.79%, and a "code-safe" crusher that reported **zero** loss until independent predicates showed it still dropped 8,371 code lines (`docs/receiving-compressor-bottleneck.md`).

Four documents in `docs/` are retractions of the author's own claims. No competitor does this. `Mem0` publishes 26% over OpenAI on LOCOMO; Zep publishes 75.14% and argues the benchmark is flawed; Honcho claims 60–90%. Zep's sharpest observation is that **Mem0's own paper shows a plain full-context baseline beating Mem0** (~73% vs ~68%) — the standing warning for every memory product, `memor` included.

4.2 The safety architecture

The source guard is correct, and four attempts to weaken it this session failed on evidence. Releasing guarded payloads destroys 5,230 code lines across 273 of 443 payloads, because `sed -n '1,200p' file.tsx` is a file read that merely lacks line numbers.

The system fails open almost everywhere, right for a sidecar: socket down falls back inline (`bin/memor-hook.py:80–85`), missing embedder returns the body unchanged (`memor/proxy/memory.py:115–120`).

4.3 Concurrency

WAL plus a 30-second busy timeout (`memor/store/sqlite_store.py:106`), with the motivating incident in the comment: 356 "database is locked" errors in one session under Python's 5-second default. This is what makes one store shared by several concurrently-running tools viable at all — a precondition for the thesis.

5. Other defects

5.1 Finished work stranded on unreachable tags

Four shelved tags, **1,759 insertions**, contained in no branch:

Tag	Commits	Diff
<code>shelved/temporal-validity</code>	11	19 files, +699
<code>shelved/reaffirmation-recency</code>	4	9 files, +532
<code>shelved/dashboard-service</code>	1	4 files, +352
<code>shelved/distillation-quality</code>	2	3 files, +176

`temporal-validity` matters most: the `disputes` table and `validity` column exist in the live database — `disputes` holds 0 rows, `validity` is 1.0 across all 4,061 rows, and **neither string appears anywhere in the Python source**. A fossil of a completed feature. Staleness is the known failure mode of code memory, and it compounds across tools: a memory written by one tool and read by another months later has no freshness signal at all.

5.2 Dead code and a silent kill switch

`memor/proxy/output_shaper.py` is **271 lines referenced from nowhere**.

`memor/proxy/shim.py:35–46` sets `compressor_ready = False` on any exception, and `prepare_request_body` checks that latch on entry. **One transient DB lock permanently disables compression for the life of the process**, no reset, no alert.

5.3 The benchmark measures the wrong construct

`eval-longmemeval` reports 95.0% any-hit, but scores **set membership of session IDs** (`memor/eval/longmemeval.py:111`), and the file's own docstring admits "any-hit overstates how many questions the agent could actually answer". LongMemEval is conversational personal-assistant memory ("I have a dog named Max"), not coding memory, and certainly not *cross-tool* coding memory. The one end-to-end number in the repo collapsed from **63.8% to 8.6%** when the harness was corrected to call production `recall()` (`CHANGELOG.md:39`).

6. Strategic position

Under the corrected thesis, memor's position is **stronger**, not weaker.

Anthropic has shipped context editing (2025-09-29), the memory tool (`memory_20250818`), auto-compact, and auto-memory. Against a *compression* product this is close to fatal: they report 84% token reduction on a 100-turn eval, and a user will compare that to 11.7% and stop reading.

Against a *cross-tool memory layer* it is not competitive at all, because **no model provider will ever make its memory work well inside a rival's harness**. Anthropic's memory is Claude's memory. Cursor's Memories are Cursor's. A developer using Claude Code, Codex and Cursor in one week has three silos by construction, and only a third party can unify them.

Anthropic ships the memory *protocol* and states plainly that "memory lives entirely in your application" — they specified the socket, not the appliance. What goes in, when, deduplicated how, ranked by what, and **shared with whom**, is unowned.

Every serious competitor has noticed the portability gap: claude-mem, Cognee, Honcho and Headroom all lead with multi-harness support. memor's differentiators within that field are **local-first** (against Mem0, Zep, Supermemory, Honcho, which are cloud APIs) and **falsifiable measurement** (against all of them).

The economics also stop being embarrassing. On a Max subscription, compression saves **\$0.00**. Cross-tool memory does not claim to save money; it claims to save *re-establishment*, which is felt on every tool switch regardless of billing model.

7. Ranked backlog

Ranked by (impact on the shared-layer thesis × confidence).

P0 — Make cross-tool recall measurable

What. Extend the feedback loop past Claude. `memor/daemon.py:372` skips every non-Claude session, so `analyze_session_feedback` never adjudicates a jcode, cursor, goose or kimi recall. Transcript parsers for these agents already exist (`memor/ingest/jcode.py` , `goose.py` , `kimi.py`); the work is wiring them into the feedback path.

Why first. All 37 cross-tool recalls are `pending` . Until this lands, every statement about the product's core capability is unfalsifiable — including the favourable ones in this document. The project's own history says an unfalsifiable claim is worth nothing: seven collapsed this session.

Success metric. Cross-tool `used / rejected` verdicts become non-zero, and the cross-tool use rate is comparable against the same-tool 100%-of-judged baseline.

Effort. Small-to-moderate. Parsers exist; the loop is one guard clause and a transcript-path resolution per agent.

Risk. Low. It adds measurement, changes no retrieval behaviour.

P1 — Express the relevance gate in cosine, then tune it on evidence

What. Declare `distance_metric=cosine` on `vec_artifacts`, or convert with `cos = 1 - L22/2`, so the constant means what its comment says. Then decide the floor from the precision/recall table in §3.2 rather than by feel.

Why. Two reasons, and the second is the smaller one.

First, correctness of reasoning: `min_similarity = -0.05` currently denotes cosine 0.449, so nobody can reason about it, port it to another embedder, or change it safely. That is a latent hazard every time the embedding model moves.

Second, recall: moving the cut from 0.449 to ~0.40 admits **67% of on-topic candidates instead of 41%**, costing 16 additional false admits on the probe set (85% precision, down from 97%). That is the specific trade available.

Important qualification. The measured table shows the existing gate is close to precision-optimal for `potion-base-8M`. This is a units bug and a modest recall opportunity, **not** the cause of the 40.9% zero-hit rate — see §3.3 for the more likely cause. I over-claimed this in an earlier draft on the strength of the conversion table alone, before probing the live store.

Success metric. Zero-hit rate on stores with >2,000 artifacts falls from 32.1%, **and** the P0 cross-tool use rate does not fall. Both must hold.

Effort. Small in code; needs a `vec_artifacts` re-index migration.

Risk. Real. Chroma's context-rot work (2025-07-14, 18 models, 194,480 calls) finds distractor harm is the dominant degradation lever and grows with context length, so the 16 extra false admits are not free. **Do not ship before P0 can measure it.**

P2 — A write-side curation filter

What. Refuse to store what the repo already answers. Prefer rationale, failed attempts, corrections, preferences.

Why. 46.3% of memory tokens are ephemeral scaffolding; 88% of artifacts are never recalled; recall already selects for rationale at 52.4%. Anthropic shipped exactly this filter. It also improves the P1 risk profile: a cleaner store means a wider gate admits knowledge rather than noise.

Success metric. Never-recalled share falls from 88%; rationale share of stored tokens rises from 44.8%.

Effort. Small-to-moderate; `_signal_score()` (`memor/ingest/claude_code.py:82`) is the hook point.

P3 — Cross-tool acceptance test

What. An end-to-end test: write a decision in tool A, recall it in tool B, assert it is served and used. Run it for each supported pair.

Why. The product's headline capability has no acceptance test. 28.7% is measured from incidental production data, not from anything guaranteeing the path keeps working. Nine integration surfaces across three mechanisms means silent breakage is the default failure mode.

Effort. Moderate.

P4 — Unshelve temporal validity

What. Restore `shelved/temporal-validity` (+699) or delete the orphaned schema.

Why. Staleness compounds in a shared layer: a memory written by one tool and read by another months later has no freshness signal. The work is done.

P5 — Constraint pinning (was P0 in the first draft)

What. Detect user-issued constraints, persist them, re-inject after compaction.

Why it moved. Measured on this machine, **≥36.9% of the user's own typed instructions are lost to compaction** (433 of 1,172 across 63 events), and 81.5% of sessions over 2MB compact. The literature is stronger still: COMPINT (arXiv:2608.11242) measures 17% constraint retention and closes it to >90% with a *sidecar*; ConstraintRot (arXiv:2606.22528) shows violations going 0%→30% purely from compaction, restored to 0% by pinning.

This is a real and valuable feature, and under the compression framing it was correctly P0. Under the shared-memory framing it is adjacent: it improves memory *within* a session rather than *between* tools. Build it after the core thesis is measurable.

Caveat on the 36.9%. It is a lower bound. The retention test counts a constraint as surviving when 60% of its content words appear in the summary, and inspection shows that fires on stopwords where the summary merely discusses the topic. A first pass also counted 21,421 "constraints" that were skill-file boilerplate; restricting to human-typed turns cut it to 1,172.

P6 — Repair operational faults

Remove or reset the `compressor_ready` latch (`memor/proxy/shim.py:35–46`); delete `memor/proxy/output_shaper.py` (271 dead lines); extract `memor/install/` from `memor/cli.py` (1,871 lines, install logic for seven agents at 701–1102).

8. What not to build

- **More compression.** Four independent attacks measured negative in one day; the remaining 4.04% is source code. `79702be` records this.
 - **A better content classifier.** Worth 0.03%–4.04% depending only on which compressor receives the release; never the bottleneck.
 - **More integration surfaces.** Nine agents, three mechanisms, two non-identical support matrices. Depth on the pairs that are actually used beats breadth — and P3 says the existing pairs are not even tested.
-

9. Open questions

1. **Do recalls replace derivations?** The 13× arithmetic (469 tokens injected vs ~6,000 to re-read a file) assumes the recall averts the read. Unmeasured, and it is the product's core economic claim.
 2. **Does cross-tool recall help as much as same-tool?** P0 exists to answer this. It could plausibly be *better* (the other tool's knowledge is genuinely new) or *worse* (different conventions, stale context).
 3. **Is 28.7% good?** No baseline exists. It could be near a ceiling set by how often two tools genuinely need the same knowledge.
 4. **Would fixing the gate improve answers or just add distractors?** The measured trade is 41%→67% recall for 97%→85% precision. Which side wins is unknown without P0.
 5. **Is the local ≥36.9% constraint loss representative?** One user, one harness, 63 events. COMPINT reports far worse across compactors.
-

Appendix A: the compression finding, for completeness

Compression is honest, finished, and capped. On the local corpus the log crusher earns **88% of all realised savings** at a 70.2% rate; everything else combined is 1.2%. Four attacks on the remaining 4.04% all measured negative (docs/receiving-compressor-bottleneck.md): a middle-ground compressor is worth 0.52% against an existing 0.21%; releasing guarded payloads destroys 5,230 code lines; a command allowlist cannot see `find -exec cat` or heredocs; and pinning code lines reported zero loss only because it was measured with the pinning regex.

The one change that survived was lowering the floor from 2,000 to 1,000 characters (+0.57% of tool-result tokens, no answer-critical line lost).

Under the shared-memory thesis, compression's role is clear and worth keeping: **it funds the memory layer**. It saves 2.45M tokens against recall's 1.15M cost. That framing is defensible in a way that "11.7% saved" is not.

Appendix B: what was measured, and how

Claim	Method
Store is tool-agnostic	<code>Scope</code> has no agent field (<code>types.py:34–46</code>); grepped retrieval path for agent filters
5.6% / 28.7% cross-tool	Joined <code>recall_outcomes</code> → <code>recall_log</code> → <code>artifacts</code> , comparing reader agent to writer source
All cross-tool recalls pending	Same join, grouped by <code>outcome</code>
Feedback is Claude-only	<code>memor/daemon.py:367–372</code>
L2-vs-cosine gate	Orthogonal unit vectors through <code>sqlite-vec</code> return $\sqrt{2}$, confirming L2; table from L2 = $\sqrt{(2-2\cos)}$
Gate precision/recall	5 on-topic queries (150 candidates) vs 4 nonsense (112) against the live store, converted to true cosine
Embedder is not the problem	Matched vs nonsense pairs under true cosine: 0.573 separation on <code>potion-base-8M</code>
Token ledger	<code>sum(tokens_injected)</code> from <code>recall_log</code> vs <code>proxy_savings</code> before/after
46.3% ephemeral / 52.4% rationale	Regex classification over all 6,948 memory artifacts
≥36.9% constraint loss	Human-typed constraints from turns preceding 63 compactions, tested for 60% content-word survival; boilerplate excluded
81.5% of >2MB sessions compact	2,465 transcripts scanned for compaction markers, bucketed by size
Shelved work	<code>git tag</code> , <code>git branch --contains</code> , <code>git diff --shortstat</code>

Research agents: `arch-map2` , `memory-qual2` , `landscape` , `product-pos` , `ctx-frontier` . Full reports in `docs/competitive-landscape-2026-08.md` and `docs/research/context-quality-2026-08.md` .